07 / SLIDE 001 Processi asincroni e parallelismo

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 1 di 18

Spiegazione per studiare

La lezione esamina come un Web server gestisce più attività, distinguendo esecuzione sincrona, asincrona, concorrente e parallela. Introduce anche framework, gateway Python, processi, thread e richieste AJAX.

Definizioni, passaggi ed esempi presenti nella slide

Processi Asincroni e Parallelismo

SLIDE ORIGINALE · diagrammi, tabelle e codice





Processi Asincroni e Parallelismo

07 / SLIDE 002 Gestione di utenti multipli e concorrenza

Lezione: Processi asincroni e parallelismo
Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 2 di 18

Spiegazione per studiare

Con molti utenti, le metriche da osservare includono latenza fra richiesta e risposta e throughput, ossia quantità di dati trasmessi nel tempo. L'obiettivo dei meccanismi concorrenti è ridurre i periodi in cui un'attività in attesa impedisce di servire altre richieste.

Definizioni, passaggi ed esempi presenti nella slide

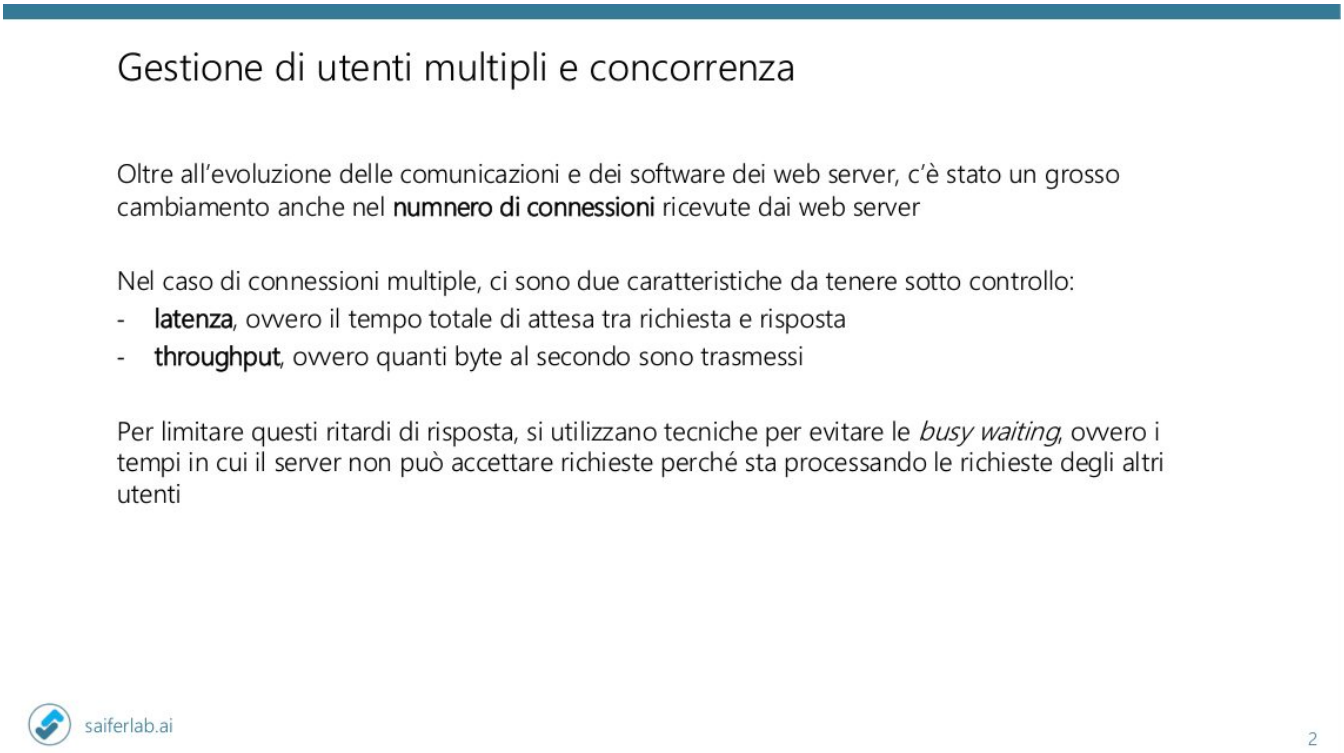
Oltre all’evoluzione delle comunicazioni e dei software dei web server, c’è stato un grosso cambiamento anche nel numnero di connessioni ricevute dai web server

Nel caso di connessioni multiple, ci sono due caratteristiche da tenere sotto controllo:

- latenza, ovvero il tempo totale di attesa tra richiesta e risposta
- throughput, ovvero quanti byte al secondo sono trasmessi

Per limitare questi ritardi di risposta, si utilizzano tecniche per evitare le busy waiting, ovvero i tempi in cui il server non può accettare richieste perché sta processando le richieste degli altri utenti

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 003 Gestione di utenti multipli e concorrenza

Lezione: Processi asincroni e parallelismo
Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 3 di 18

Spiegazione per studiare

Nel modello sincrono un'operazione attende la precedente e può bloccare l'avanzamento delle successive. Con l'elaborazione asincrona si possono avviare attività che attendono I/O senza occupare inutilmente il flusso di esecuzione, tornando al risultato quando è disponibile.

Definizioni, passaggi ed esempi presenti nella slide

La normale esecuzione in Python è sincrona: i comandi vengono eseguiti in ordine di arrivo e il comando successivo non inizia finché non è iniziato il precedente

• esecuzione bloccante, non fa altro fino a che non termina l'esecuzione

Nel caso della gestione di utenti multipli, il metodo asincrono è molto meglio

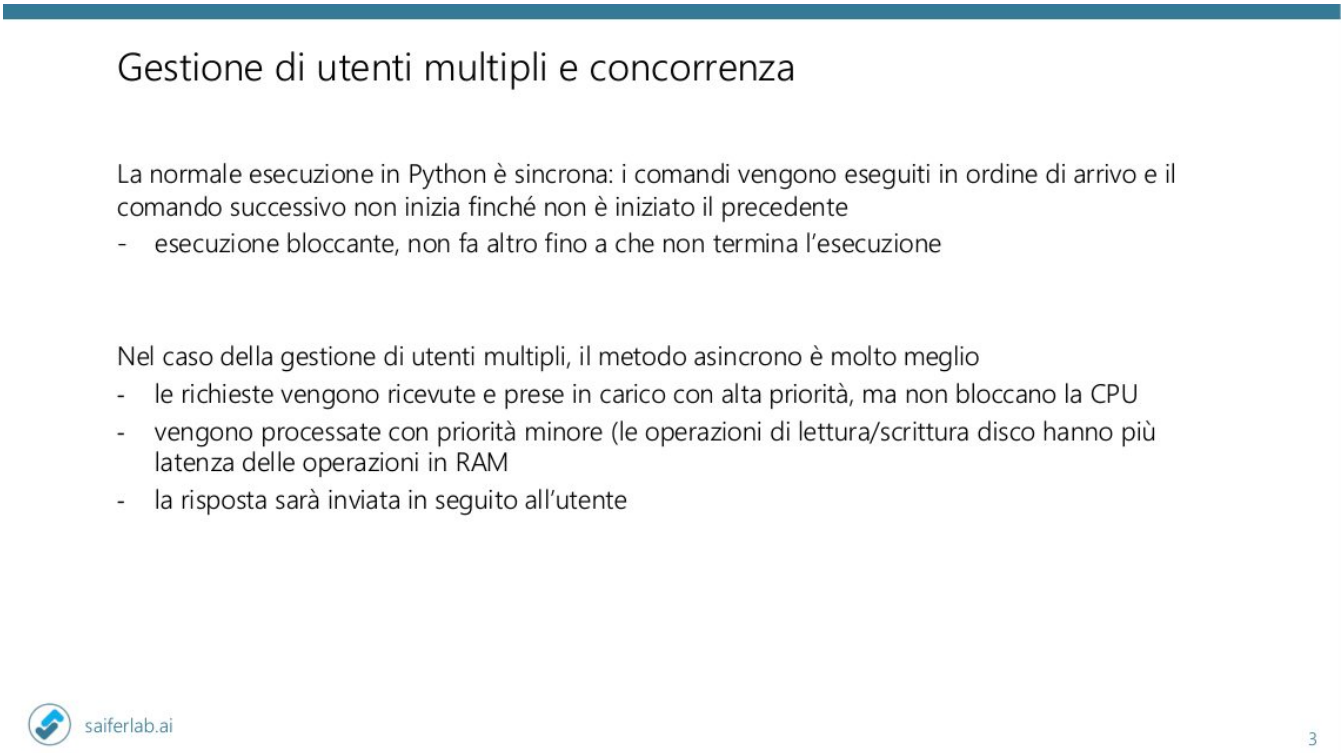
• le richieste vengono ricevute e prese in carico con alta priorità, ma non bloccano la CPU

• vengono processate con priorità minore (le operazioni di lettura/scrittura disco hanno più

latenza delle operazioni in RAM

• la risposta sarà inviata in seguito all'utente

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 004 Esempi di utilizzo dei processi asincroni

Lezione: Processi asincroni e parallelismo
Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 4 di 18

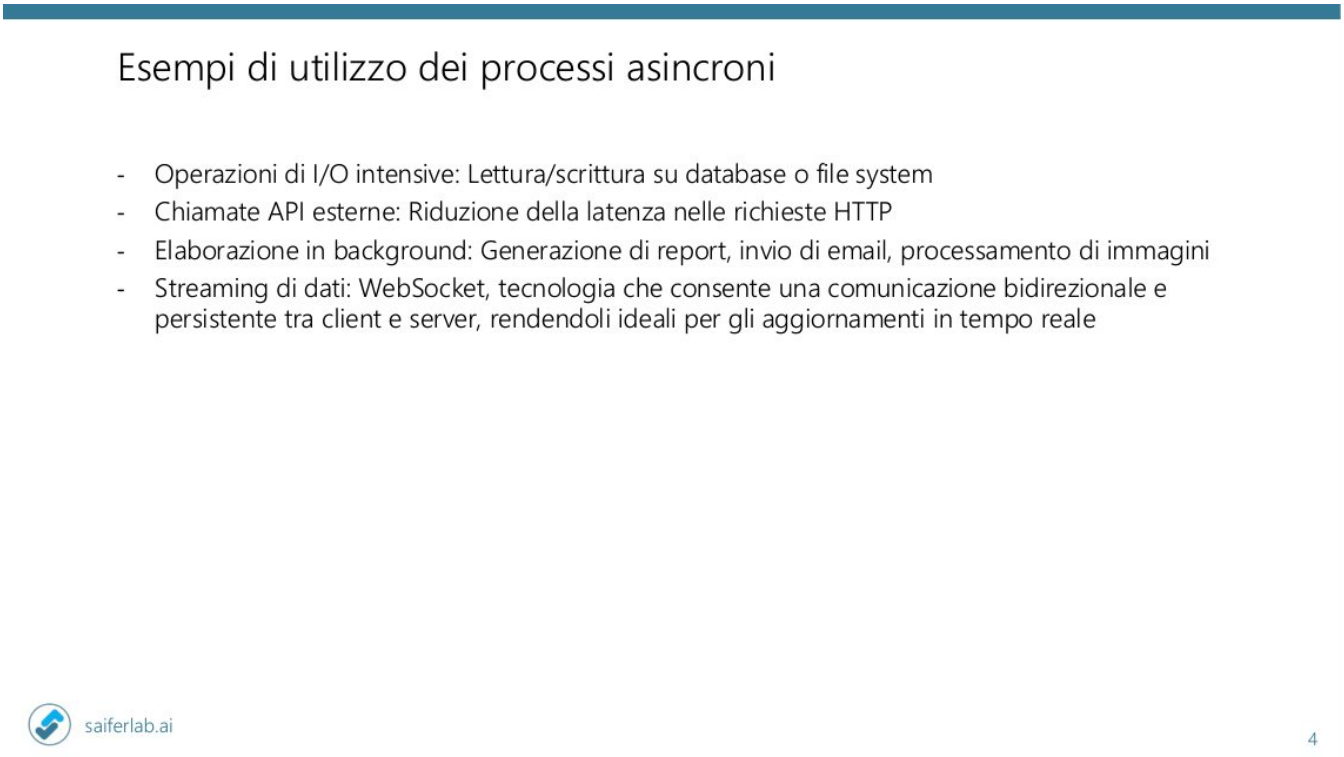
Spiegazione per studiare

L'asincronia è utile soprattutto nelle operazioni lente di input/output: letture del database, accesso ai file, chiamate HTTP, invio di email e flussi in tempo reale. Il beneficio deriva dalla possibilità di usare il tempo di attesa per gestire altre attività.

Definizioni, passaggi ed esempi presenti nella slide

- Operazioni di I/O intensive: Lettura/scrittura su database o file system
- Chiamate API esterne: Riduzione della latenza nelle richieste HTTP
- Elaborazione in background: Generazione di report, invio di email, processamento di immagini
- Streaming di dati: WebSocket, tecnologia che consente una comunicazione bidirezionale e persistente tra client e server, rendendoli ideali per gli aggiornamenti in tempo reale

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 005 L’utilizzo di framework

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 5 di 18

Spiegazione per studiare

Un framework offre componenti già integrati per sviluppare il server senza implementare da zero tutti i meccanismi di gestione delle richieste. Oltre a velocizzare lo sviluppo, può fornire strumenti e astrazioni per affrontare la concorrenza e migliorare l'organizzazione del codice.

Definizioni, passaggi ed esempi presenti nella slide

Per gestire la concorrenza, la gestione di utenti multipli, e tante altre cose, non è raccomandabile scrivere tutto il codice partendo da “zero”

Solitamente si sceglie un framework, ovvero un isieme di software e funzionalità che sono pensate per lavorare assieme per lo sviluppo accelerato

Nella pratica, tra i vantaggi ci sono:

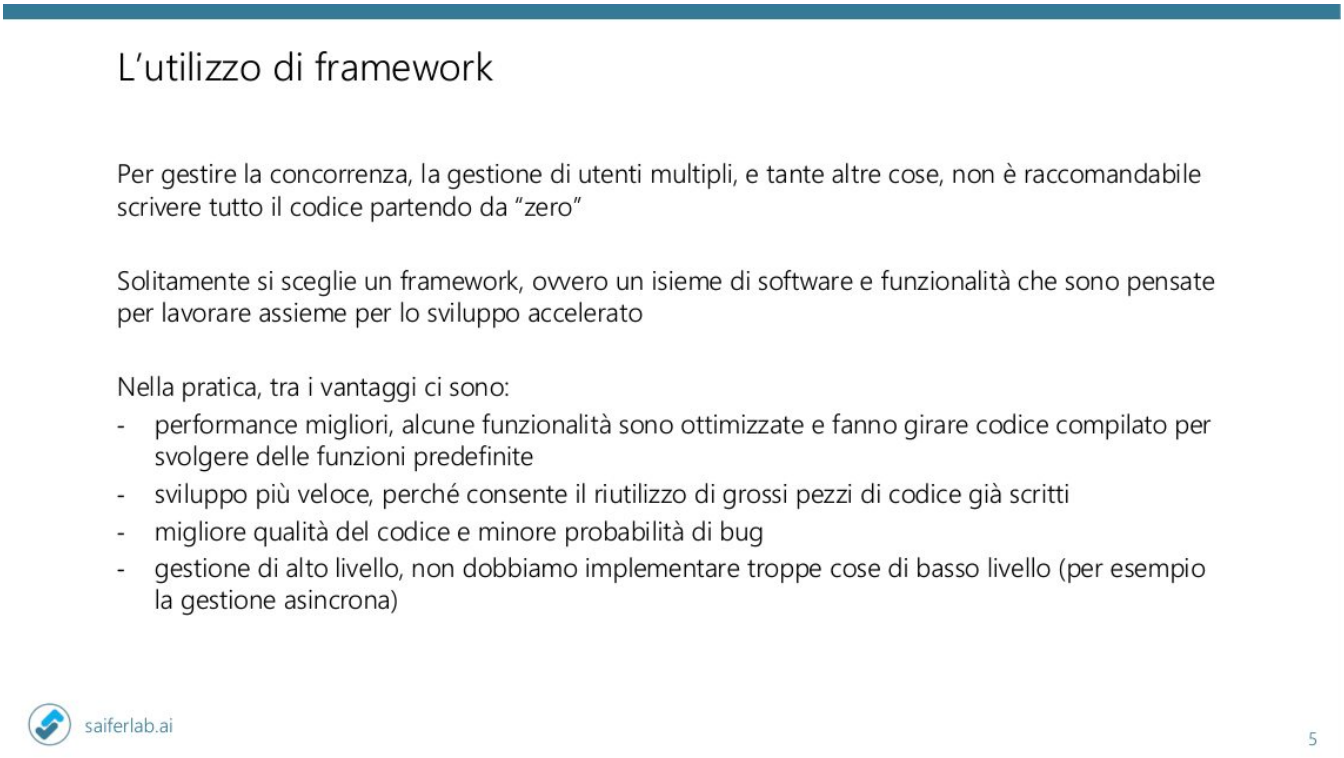
• performance migliori, alcune funzionalità sono ottimizzate e fanno girare codice compilato per svolgere delle funzioni predefinite

• sviluppo più veloce, perché consente il riutilizzo di grossi pezzi di codice già scritti

• migliore qualità del codice e minore probabilità di bug

• gestione di alto livello, non dobbiamo implementare troppe cose di basso livello (per esempio la gestione asincrona)

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 006 Come vengono implementati i processi asincroni nei web server

Lezione: Processi asincroni e parallelismo
Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 6 di 18

Spiegazione per studiare

Tra i modelli per l'asincronia la slide presenta callback, promise, `async/await`, code di lavori e programmazione a eventi. Le callback eseguono azioni quando un'operazione termina; code e task scheduler permettono invece di delegare attività di background.

Definizioni, passaggi ed esempi presenti nella slide

Programmazione basata su Callback

- Funzioni che vengono eseguite una volta completata un'operazione
- possono portare a problemi di "callback hell” (quando si annidano molte funzioni di callback

all'interno di altre, rendendo il codice difficile da leggere e mantenere) Promises e Async/Await

- Promises: Struttura che rappresenta un'operazione asincrona completata o fallita
- Async/Await: Semplifica la scrittura del codice asincrono, rendendolo più leggibile

Utilizzo di code: job Queue e Task Scheduling

- Esempi: Celery (Python), Sidekiq (Ruby), Bull (Node.js)
- Utilizzati per gestire compiti in background in modo efficiente

WebSockets e Event-Driven Programming

- WebSocket permette una comunicazione bidirezionale in tempo reale tra client e server
- Modelli event-driven come Node.js utilizzano il loop degli eventi per gestire richieste asincrone

SLIDE ORIGINALE · diagrammi, tabelle e codice

Come vengono implementati i processi asincroni nei web server

Programmazione basata su Callback

- Funzioni che vengono eseguite una volta completata un'operazione
- possono portare a problemi di "callback hell" (quando si annidano molte funzioni di callback all'interno di altre, rendendo il codice difficile da leggere e mantenere)

Promises e Async/Await

- Promises: Struttura che rappresenta un'operazione asincrona completata o fallita
- Async/Await: Semplifica la scrittura del codice asincrono, rendendolo più leggibile

Utilizzo di code: job Queue e Task Scheduling

- Esempi: Celery (Python), Sidekiq (Ruby), Bull (Node.js)
- Utilizzati per gestire compiti in background in modo efficiente

WebSockets e Event-Driven Programming

- WebSocket permette una comunicazione bidirezionale in tempo reale tra client e server
- Modelli event-driven come Node.js utilizzano il loop degli eventi per gestire richieste asincrone

07 / SLIDE 007 Web applications e componenti di connessione con i web server

Lezione: Processi asincroni e parallelismo
Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 7 di 18

Spiegazione per studiare

Tra il Web server e l'applicazione Python può esserci un'interfaccia gateway con compiti di raccordo. Lo schema mostra browser, server gateway e framework. Questa separazione evita che l'applicazione sia legata direttamente ai dettagli della comunicazione di rete.

Definizioni, passaggi ed esempi presenti nella slide

Per connettere il web server con l’applicaizone web che vedrà l’utente, ci serve un’interfaccia che metta in comunicazione questi. due pezzi

Questa interfaccia prende il nome di Interfaccia Gateway e consente di separare l’applicazione dal framework usato Interfaccia Gateway

- Traduzione di protocolli
- Routing e smistamento delle richieste
- Sicurezza e autenticazione
- Bilanciamento del carico

Questo significa che cambiando il framework, si può scrivere la stessa applicazione senza modificarne l’aspetto e il modo in cui gli utenti ci devono interagire

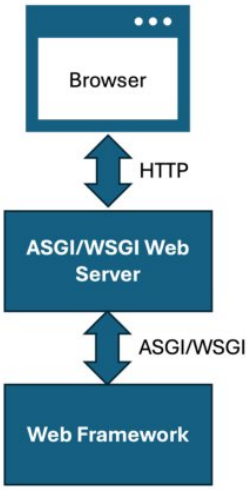
SLIDE ORIGINALE · diagrammi, tabelle e codice

Web applications e componenti di connessione con i web server

Per connettere il **web server** con l’**applicaizone web** che vedrà l’utente, ci serve un’interfaccia che metta in comunicazione questi. due pezzi

Questa interfaccia prende il nome di Interfaccia Gateway e consente di separare l’applicazione dal framework usato
Interfaccia Gateway
- Traduzione di protocolli
- Routing e smistamento delle richieste
- Sicurezza e autenticazione
- Bilanciamento del carico

Questo significa che cambiando il framework, si può scrivere la stessa applicazione senza modificarne l’aspetto e il modo in cui gli utenti ci devono interagire



saiferlab.ai

7

07 / SLIDE 008 Web Server Gateway Interface (WSGI)

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 8 di 18

Spiegazione per studiare

WSGI è uno standard di interfacciamento tra server e applicazioni Python usato da framework come Flask e Django. Nelle dispense è messo in relazione con la gestione tradizionale delle richieste e con i limiti quando occorre trattare un elevato numero di connessioni asincrone.

Definizioni, passaggi ed esempi presenti nella slide

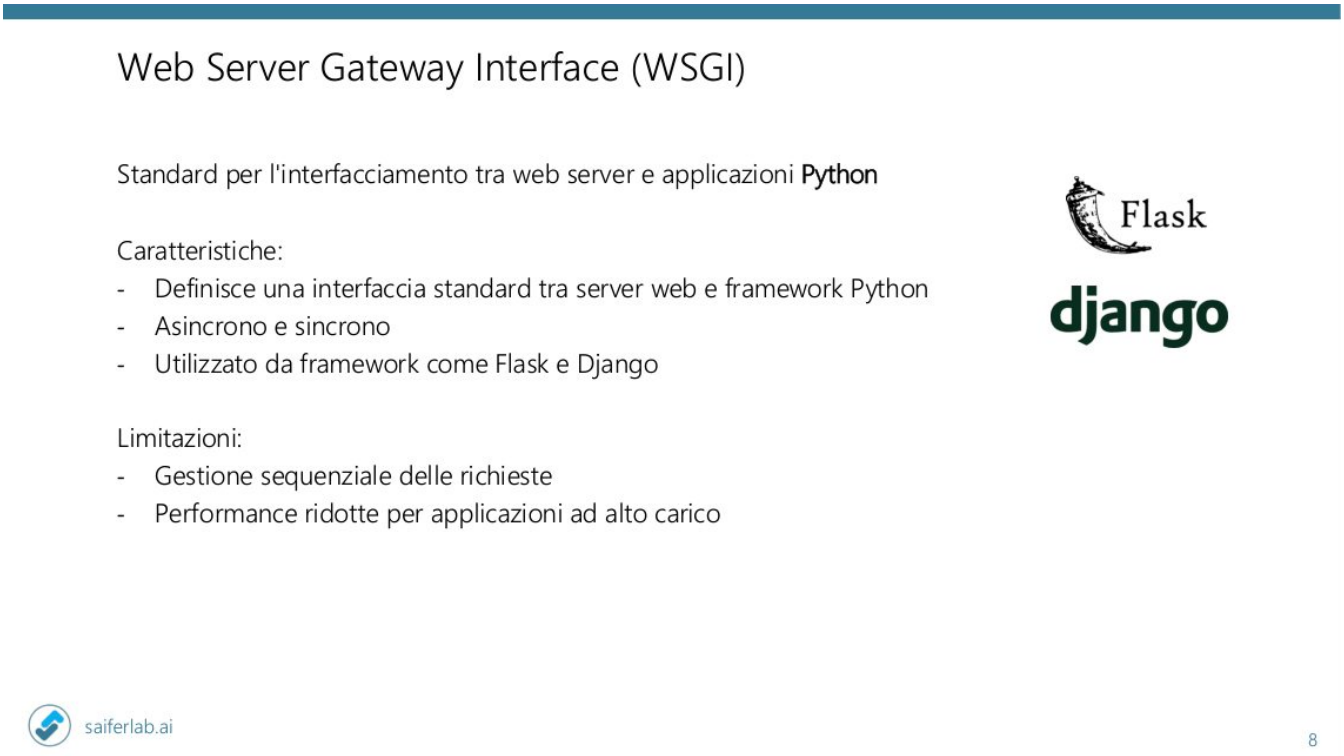
Standard per l'interfacciamento tra web server e applicazioni Python

```
Caratteristiche:
-
Definisce una interfaccia standard tra server web e framework Python
-
Asincrono e sincrono
-
Utilizzato da framework come Flask e Django
```

Limitazioni:

- Gestione sequenziale delle richieste
- Performance ridotte per applicazioni ad alto carico

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 009 Asynchronous Server Gateway Interface (ASGI)

Lezione: Processi asincroni e parallelismo
Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 9 di 18

Spiegazione per studiare

ASGI supporta applicazioni Python asincrone e protocolli come WebSocket. Si abbina a framework quali FastAPI e Starlette e permette di sfruttare `async/await` per gestire più richieste quando esistono operazioni non bloccanti.

Definizioni, passaggi ed esempi presenti nella slide

Evoluzione moderna di WSGI per applicazioni asincrone

Vantaggi principali:

- Supporto nativo per async/await
- Gestione contemporanea di più richieste
- Migliori performance per applicazioni I/O intensive
- Compatibile con WebSocket (connessione bidirezionale full-duplex, ogni parte può inviare e ricevere dati nello stesso momento) e server push (server invia dati al client senza richiesta esplicita)
- Framework principali: FastAPI, Starlette

Migliore di WSGI perché:

- Maggiore scalabilità
- Gestione efficiente delle connessioni e prestazioni superiori in ambienti ad alto carico
- Supporto a protocolli moderni

SLIDE ORIGINALE · diagrammi, tabelle e codice

Asynchronous Server Gateway Interface (ASGI)

Evoluzione moderna di WSGI per **applicazioni asincrone**



Vantaggi principali:

- Supporto nativo per async/await
- Gestione contemporanea di più richieste
- Migliori performance per applicazioni I/O intensive
- Compatibile con WebSocket (connessione bidirezionale full-duplex, ogni parte può inviare e ricevere dati nello stesso momento) e server push (server invia dati al client senza richiesta esplicita)
- Framework principali: **FastAPI**, Starlette

Migliore di WSGI perché:

- Maggiore scalabilità
- Gestione efficiente delle connessioni e prestazioni superiori in ambienti ad alto carico
- Supporto a protocolli moderni



saiferlab.ai

9

07 / SLIDE 010 Parallelismo, concorrenza, e supporto asincono

Lezione: Processi asincroni e parallelismo
Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 10 di 18

Spiegazione per studiare

Parallelismo e concorrenza non sono sinonimi. Nel parallelismo parti di lavoro vengono eseguite su più risorse di calcolo; nella concorrenza diversi task avanzano con un'organizzazione che alterna l'esecuzione e limita gli sprechi dovuti all'attesa.

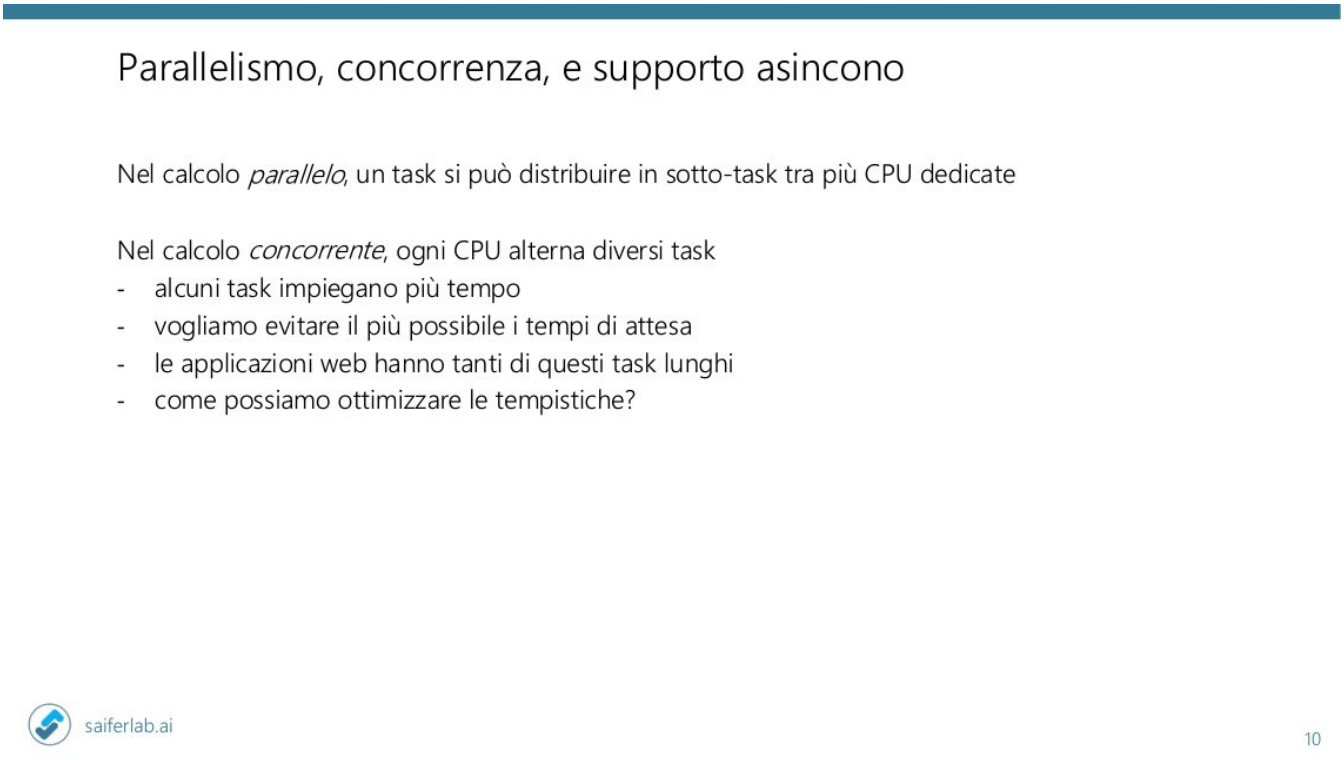
Definizioni, passaggi ed esempi presenti nella slide

Nel calcolo parallelo, un task si può distribuire in sotto-task tra più CPU dedicate

Nel calcolo concorrente, ogni CPU alterna diversi task

- alcuni task impiegano più tempo
- vogliamo evitare il più possibile i tempi di attesa
- le applicazioni web hanno tanti di questi task lunghi
- come possiamo ottimizzare le tempistiche?

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 011 Calcolo parallelo e distribuito

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 11 di 18

Spiegazione per studiare

Quando un'applicazione è grande, il lavoro può essere suddiviso in task indipendenti eseguiti su più CPU oppure su macchine diverse. La slide introduce così il passaggio dal calcolo parallelo su un computer al calcolo distribuito.

Definizioni, passaggi ed esempi presenti nella slide

Quando l’applicazione diventa complicata, o quando si prevedono molti utenti, è consigliato spezzettarla in più task che possono essere svolti indipendentemente

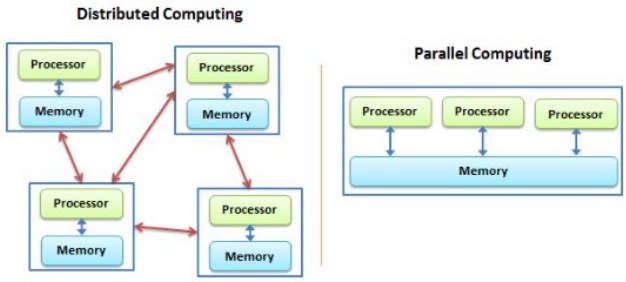
- in diverse CPU su una sola macchina (calcolo parallelo)
- o addirittura su più macchine (calcolo distribuito)

SLIDE ORIGINALE · diagrammi, tabelle e codice

Calcolo parallelo e distribuito

Quando l’applicazione diventa complicata, o quando si prevedono molti utenti, è consigliato spezzettarla in più task che possono essere svolti indipendentemente

- in diverse CPU su una sola macchina (calcolo parallelo)
- o addirittura su più macchine (calcolo distribuito)



saiferlab.ai

11

07 / SLIDE 012 I processi nel sistema operativo

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 12 di 18

Spiegazione per studiare

Il sistema operativo assegna risorse come CPU, memoria e dispositivi ai programmi in esecuzione. Un processo rappresenta un'istanza di programma con il proprio contesto di esecuzione: conoscere questo concetto serve a capire come il server usa le risorse disponibili.

Definizioni, passaggi ed esempi presenti nella slide

Nel sistema operativo le risorse (memoria, CPU, device, scheda di rete, ...) vengono “prenotate”

Ogni programma viene eseguito in uno o più processi che utilizzano queste risorse

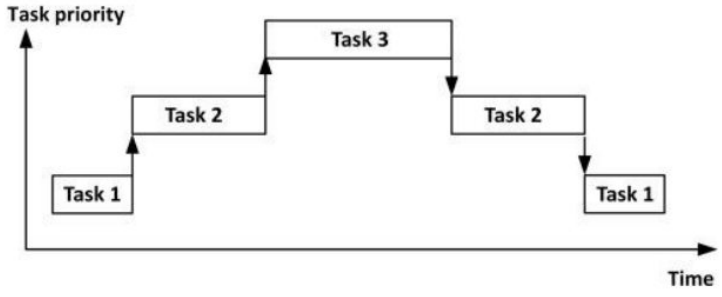
- molti sistemi usano l’allocazione “interrompibile” (preemptive)
- i processi hanno priorità dinamiche
- processi a bassa priorità possono essere sospesi a favore di processi ad alta priorità

SLIDE ORIGINALE · diagrammi, tabelle e codice

I processi nel sistema operativo

Nel sistema operativo le risorse (memoria, CPU, device, scheda di rete, ...) vengono “prenotate”

Ogni programma viene eseguito in uno o più processi che utilizzano queste risorse
- molti sistemi usano l’allocazione “**interrompibile**” (preemptive)
- i processi hanno priorità dinamiche
- processi a bassa priorità possono essere **sospesi** a favore di processi ad alta priorità



saiferlab.ai

12

07 / SLIDE 013 I thread del sistema operativo

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 13 di 18

Spiegazione per studiare

I thread sono flussi di esecuzione all'interno di un processo e condividono parte delle sue risorse. Il confronto con i processi riguarda isolamento, memoria e costo di gestione. La scelta del modello di concorrenza deve tenere conto delle operazioni svolte.

Definizioni, passaggi ed esempi presenti nella slide

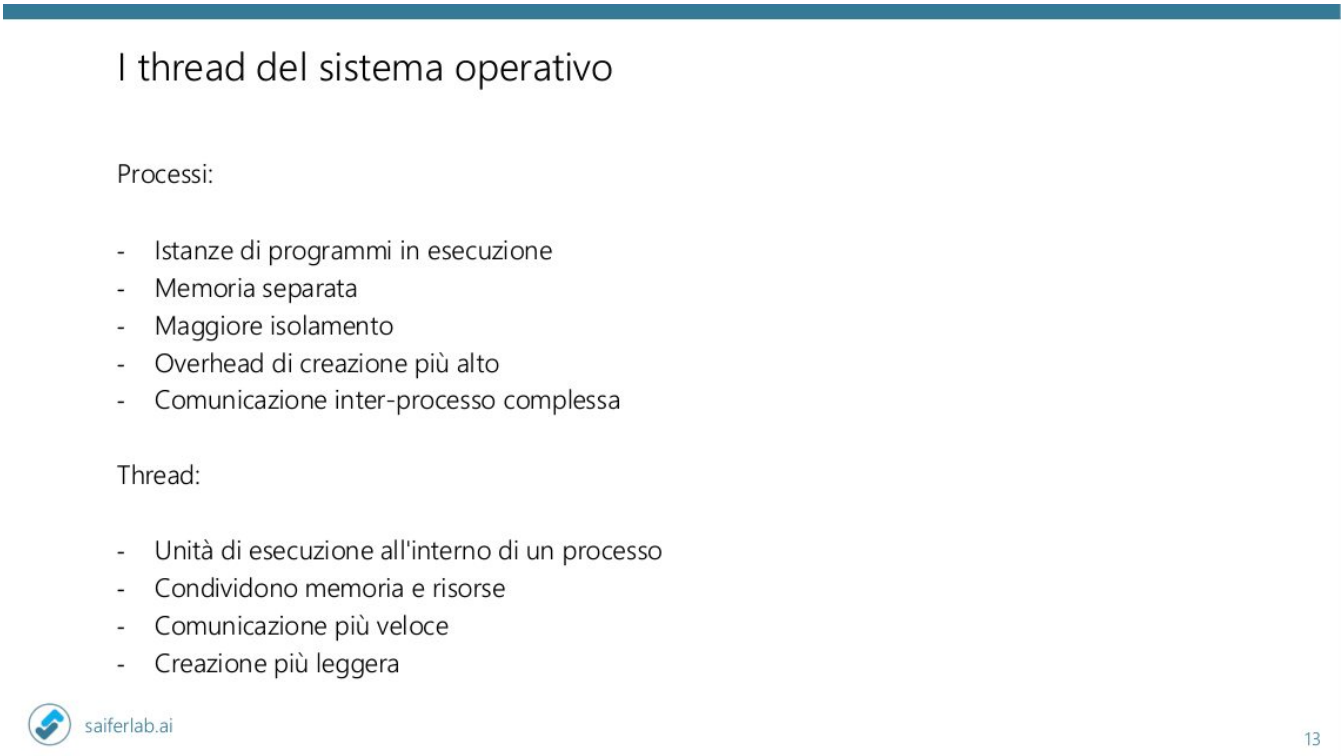
Processi:

- Istanze di programmi in esecuzione
- Memoria separata
- Maggiore isolamento
- Overhead di creazione più alto
- Comunicazione inter-processo complessa

Thread:

- Unità di esecuzione all'interno di un processo
- Condividono memoria e risorse
- Comunicazione più veloce
- Creazione più leggera

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 014 Esempio di thread asincroni

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 14 di 18

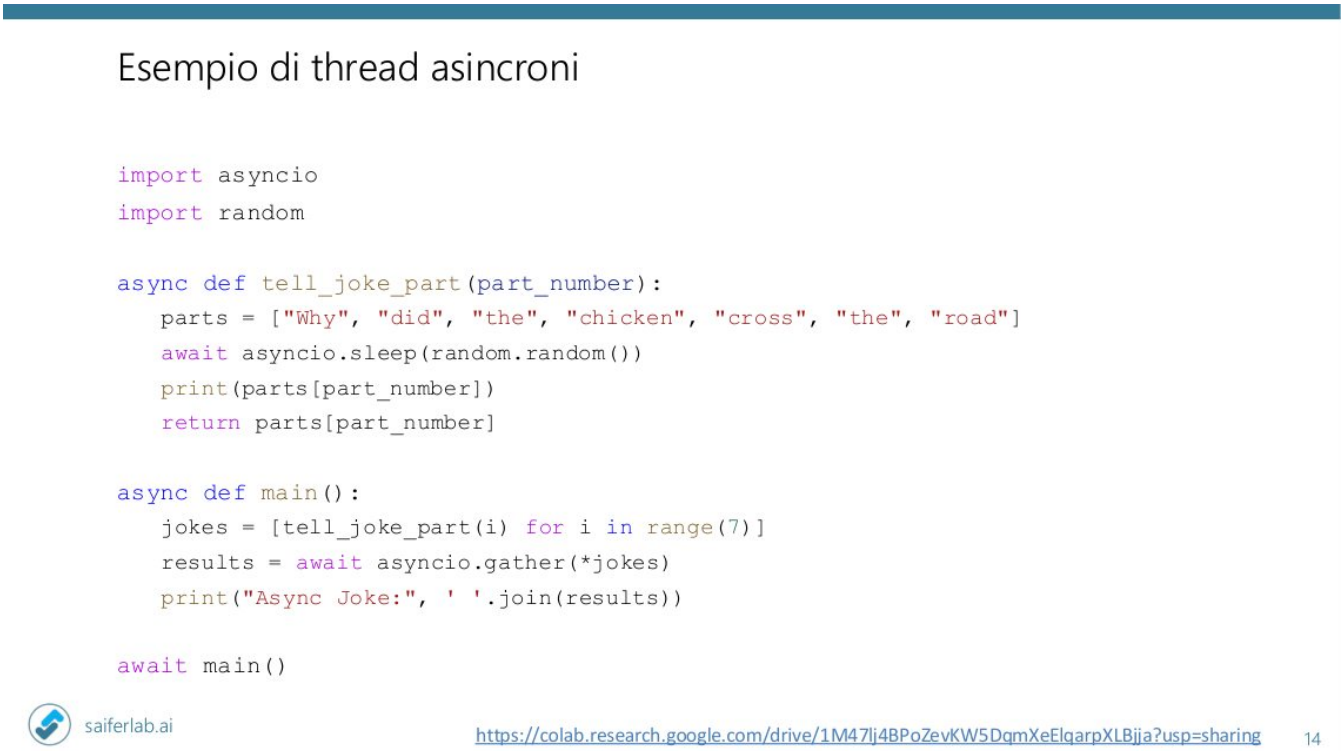
Spiegazione per studiare

L'esempio Python usa `asyncio` per mostrare come si definiscono attività asincrone. L'attenzione va posta sul modo in cui le coroutine vengono avviate e su dove il controllo viene ceduto, non soltanto sull'ordine con cui appaiono le istruzioni nel codice.

Definizioni, passaggi ed esempi presenti nella slide

```
  import asyncio
  import random
  async def tell_joke_part(part_number):
parts = ["Why", "did", "the", "chicken", "cross", "the", "road"]
  await asyncio.sleep(random.random())
print(parts[part_number])
  return parts[part_number]
  async def main():
jokes = [tell_joke_part(i) for i in range(7)]
results = await asyncio.gather(*jokes)
print("Async Joke:", ' '.join(results))
  await main()
```

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 015 Esempio di thread asincroni

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 15 di 18

Spiegazione per studiare

Le parole chiave `async` e `await` identificano funzioni asincrone e i punti in cui si attende un risultato senza bloccare necessariamente l'intero programma. La slide completa l'esempio di esecuzione concorrente e aiuta a interpretare i tempi di avanzamento dei task.

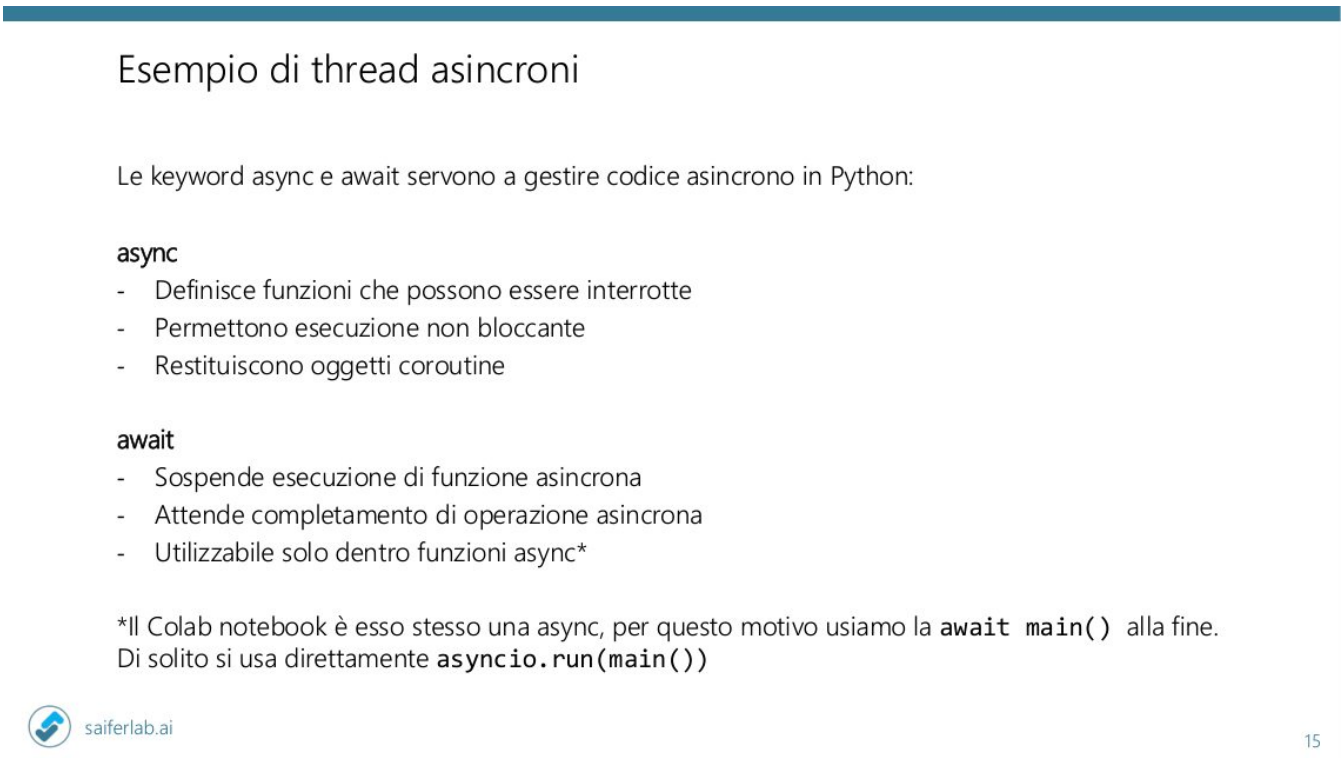
Definizioni, passaggi ed esempi presenti nella slide

Le keyword async e await servono a gestire codice asincrono in Python:

```
async
-
Definisce funzioni che possono essere interrotte
-
Permettono esecuzione non bloccante
-
Restituiscono oggetti coroutine
await
-
Sospende esecuzione di funzione asincrona
-
Attende completamento di operazione asincrona
-
Utilizzabile solo dentro funzioni async*
```

*Il Colab notebook è esso stesso una async, per questo motivo usiamo la await main() alla fine. Di solito si usa direttamente asyncio.run(main())

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 016 Thread multipli nei web server

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 16 di 18

Spiegazione per studiare

Un Web server multithread può servire più richieste mediante flussi di esecuzione separati. Il modello è utile da confrontare con server asincroni: i due meccanismi organizzano in modo diverso il lavoro concorrente e l'impiego delle risorse.

Definizioni, passaggi ed esempi presenti nella slide

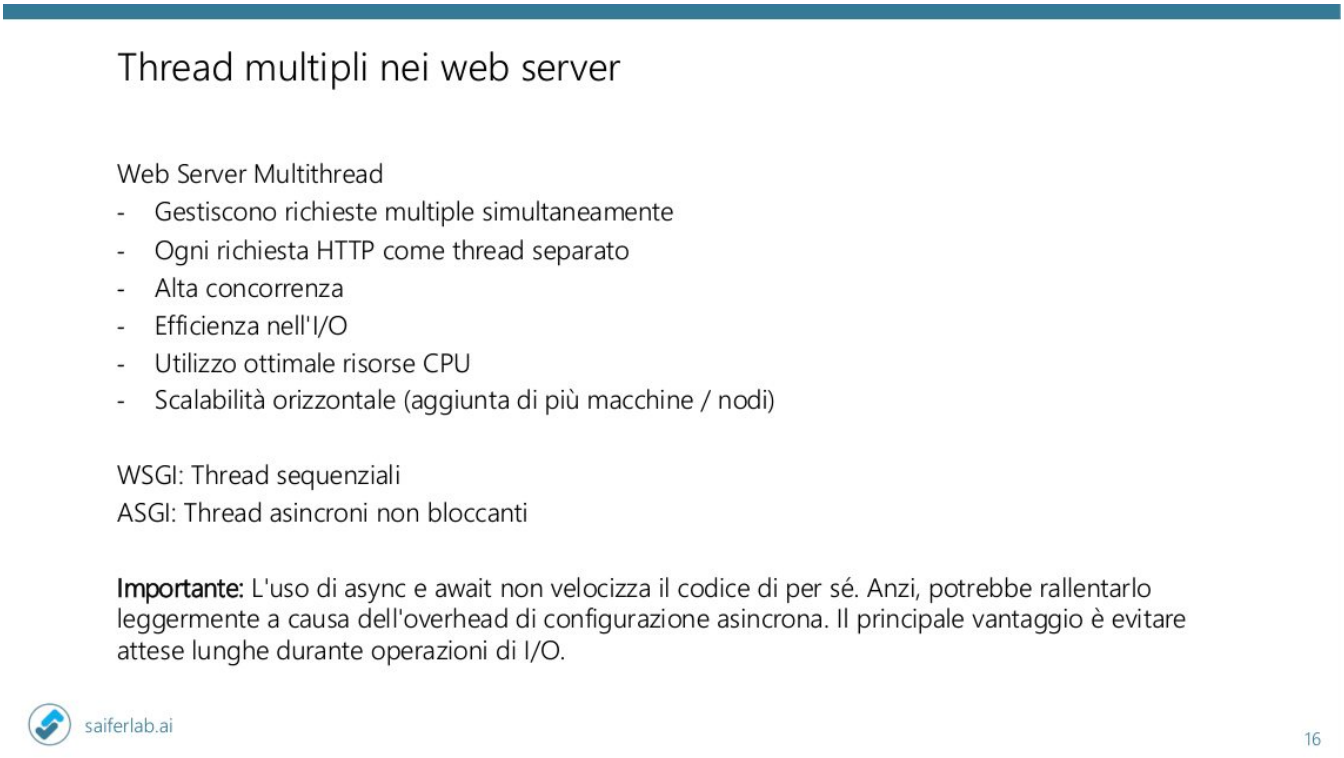
Web Server Multithread

- Gestiscono richieste multiple simultaneamente
- Ogni richiesta HTTP come thread separato
- Alta concorrenza
- Efficienza nell'I/O
- Utilizzo ottimale risorse CPU
- Scalabilità orizzontale (aggiunta di più macchine / nodi)

WSGI: Thread sequenziali ASGI: Thread asincroni non bloccanti

```
Importante: L'uso di async e await non velocizza il codice di per sé. Anzi,
potrebbe rallentarlo
leggermente a causa dell'overhead di configurazione asincrona. Il principale
vantaggio è evitare
attese lunghe durante operazioni di I/O.
```

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 017 Asynchronous JavaScript And XML (AJAX)

Lezione: Processi asincroni e parallelismo
Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 17 di 18

Spiegazione per studiare

AJAX permette a una pagina Web di richiedere dati e aggiornare parti dell'interfaccia senza ricaricare necessariamente l'intero documento. L'interazione fra JavaScript e server avviene in modo asincrono e può usare XML o altri formati di scambio dati.

Definizioni, passaggi ed esempi presenti nella slide

AJAX è un meccanismo per aggiornare le pagine web in modo asincrono, tramite lo scambio dati con un web server

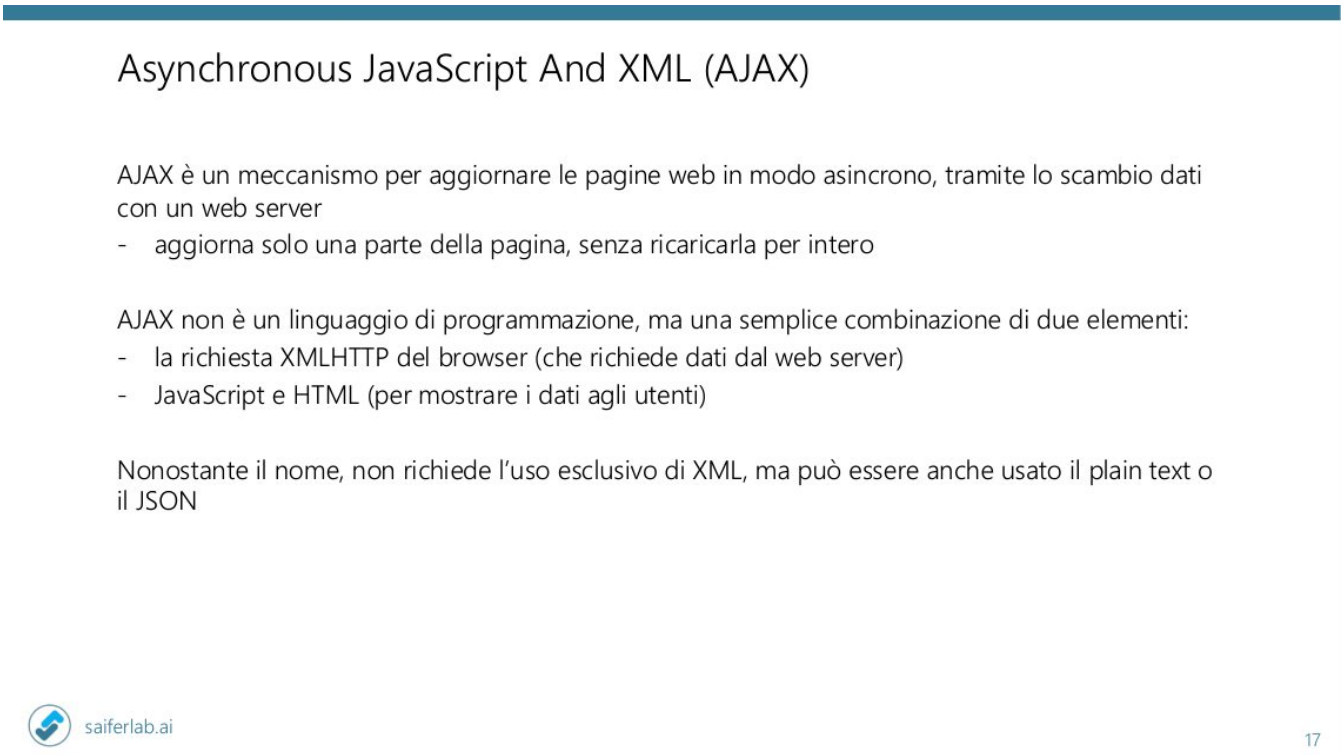
- aggiorna solo una parte della pagina, senza ricaricarla per intero

AJAX non è un linguaggio di programmazione, ma una semplice combinazione di due elementi:

- la richiesta XMLHTTP del browser (che richiede dati dal web server)
- JavaScript e HTML (per mostrare i dati agli utenti)

Nonostante il nome, non richiede l'uso esclusivo di XML, ma può essere anche usato il plain text o il JSON

SLIDE ORIGINALE · diagrammi, tabelle e codice



07 / SLIDE 018 Come funziona AJAX

Lezione: Processi asincroni e parallelismo

Fonte: 07_processi_asincroni_parallelismo.pdf, pagina 18 di 18

Spiegazione per studiare

Il ciclo AJAX parte da un evento nel browser, continua con una richiesta JavaScript al server e termina con l'aggiornamento dell'interfaccia quando arriva la risposta. È un'applicazione concreta della distinzione tra avvio dell'operazione e disponibilità del risultato.

Definizioni, passaggi ed esempi presenti nella slide

1. AJAX è attivato da un evento (per esempio un pulsante premuto dall’utente) 2. JavaScript crea la richiesta e la invia al server 3. il server risponde alla richiesta e la rimanda al browser 4. La risposta è letta tramite JavaScript, che aggiorna la pagina

SLIDE ORIGINALE · diagrammi, tabelle e codice

